Performance Analysis of GNNs Under Varying Model Depth and Hyperparameters

Noel Sarban and Timothy Besada

***Abstract*—This project aimed to develop a solid understanding of the theory behind Graph Neural Networks (GNNs) and gain practical experience implementing them. A small-scale literature review of GNNs was first conducted in the form of a theoretical background, followed by implementing multiple models where model design and hyperparameters were varied using PyTorch Geometric. Graph Convolutional Networks (GCNs) were specifically implemented, varying the number of message passing layers, dropout rates, and optimizer choices to observe how these decisions affected performance. Model depth negatively impacted performance, likely due to over-smoothing. The other experimental results did not reveal large performance differences across different dropout rates and optimizers, but key limitations in the methodology were identified that likely influenced these outcomes. Nevertheless, the models performed very well compared to other implementations, with the best test accuracy at 81.9%. This is close to state-of-the-art performance despite the relatively simple approach. The project highlights both the accessibility and the complexity of designing and training GNNs.**

***Sammanfattning*—Målet med detta projekt var att få en förståelse för teorin bakom Grafiska Neurala Nätverk (GNNs) samt att få praktisk erfarenhet av att implementera dem. Projektet började med en mindre litteraturstudie i form av en teoretisk bakgrund, följt av implementationen av flera modeller där designval och hyperparametrar varierades med hjälp av PyTorch Geometric. Graph Convolutional Networks (GCNs) implementerades specifikt, där antalet lager för message passing, dropout-nivåer och val av optimeringsalgoritm varierades för att undersöka hur dessa val påverkade prestandan. Modellens djup påverkade prestandan negativt, sannolikt på grund av "over-smooothing". De övriga experimentella resultaten visade inte stora prestandaskillnader mellan olika dropout-nivåer och optimeringsalgoritmer, men viktiga metodologiska begränsningar identifierades som sannolikt påverkade resultaten. Trots detta presterade modellerna mycket väl i jämförelse med andra implementationer, med den bästa "test-accuracy" på 81,9%. Detta är nära state-of-the-art-prestanda trots den relativt enkla metoden. Projektet belyser både tillgängligheten och komplexiteten i att designa och träna GNNs.**

***Index Terms*—Graph Neural Networks, Graph Convolutional Networks, Hyperparameter Tuning, Dropout, Model Depth, Deep Learning on Graphs, Empirical Evaluation**

Supervisors: Zeyang Li, Carlo Fischione

TRITA number: TRITA-EECS-EX-2025:141

I. Introduction

Neural networks have become a key tool in modern machine learning, showing impressive results across a wide range of tasks, from image recognition and language translation to game playing and recommendation systems [1]. These models work particularly well when the input data can be structured as a vector, such as for an image or a sentence.

However, for many contemporary problems, the data does not conform to such a structured format. Instead, it can often be more naturally represented as a graph. Graphs are structures composed of nodes and edges, where each edge represents a relationship or connection between two nodes. This type of structure appears in social networks, citation networks, molecules, transportation systems, and many other domains. In these settings, not only the content of individual nodes matters, but also the manner in which they are connected.

Conventional neural networks do not handle graph-structured data effectively, as they disregard the connections between elements. This limitation necessitates the development of Graph Neural Networks (GNNs). GNNs are specifically designed to operate on graphs and leverage the relationships between nodes. They enable nodes to exchange information with their neighbors through a process called message passing [2], which facilitates the network's ability to learn useful features based on both a node's individual attributes and the structural context of the surrounding graph.

For all neural networks, including GNNs, hyperparameters play a critical role in shaping model performance. Understanding their effects is essential when designing and tuning a model, as they directly influence how the network learns from data. Factors such as the number of layers, dropout rate, and choice of optimizer can significantly affect the model's ability to extract useful patterns and generalize to unseen data. As a result, performance can vary widely depending on how these hyperparameters are configured.

In this project, the basic theory of GNNs is examined, followed by the implementation of a model. Subsequently, the investigation focuses on how performance is affected by varying hyperparameters. Using the Cora dataset, a citation network of scientific papers [3], results are compared across several model configurations to better understand which choices lead to improved performance. The objective is not only to compare different models, but also to gain insight into how GNNs learn from graph-structured data and what makes them effective for this type of task.

II. Theoretical Background

Understanding how Graph Neural Networks work requires some basic knowledge of neural networks, how they are trained, and how graphs can be used in machine learning. In this section, the fundamental concepts needed to understand GNNs are introduced, as well as the specifics of GNNs and the tools employed in this project.

A. Neural Networks Foundations

In this subsection, a brief overview of neural network fundamentals is provided, model training is discussed, the importance of regularization is highlighted, and finally the optimizers used in this project are described.

Neural Networks consist of many layers connected to the previous and next layer. Each layer has a number of nodes (also called neurons), and the values for the next layer are calculated based on the previous layer's values, multiplied by a set of trainable weights, and added to a set of trainable biases [1]. Mathematically, for a layer l, if $a^{(l-1)}$ represents the output of the previous layer, the pre-activation $z^{(l)}$ for the current layer is given by:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

where $W^{(l)}$ is the weight matrix and $b^{(l)}$ is the bias vector for layer l. A common approach is to rewrite this by incorporating the bias term into the weight matrix. This is achieved by appending a 1 to the input vector $a^{(l-1)}$ (making it slightly longer), and extending $W^{(l)}$ with an extra column that corresponds to the biases. This allows the entire operation to be represented as a single matrix multiplication:

$$z^{(l)} = W^{(l)}_{\text{aug}}a^{(l-1)}_{\text{aug}}$$

where $a^{(l-1)}_{\text{aug}}$ is the augmented input vector with a 1 appended, and $W^{(l)}_{\text{aug}}$ is the augmented weight matrix.

To allow the model to learn more complex, non-linear patterns, a non-linear activation function is applied to the pre-activation before passing it to the next layer. The activated output $a^{(l)}$ is then:

$$a^{(l)} = \sigma\left(z^{(l)}\right)$$

where $\sigma(\cdot)$ denotes the activation function.

One of the most commonly used activation functions today is the Rectified Linear Unit (ReLU), defined as:

$$\text{ReLU}(x) = \max(0, x)$$

ReLU is simple and efficient: it keeps positive values unchanged while zeroing out negative values. This helps neural networks avoid problems like vanishing gradients [4], which can make training deep models difficult. In the context of Graph Neural Networks (GNNs), ReLU is particularly effective because it preserves positive aggregated features while introducing necessary non-linearity, allowing GNNs to capture richer patterns across the graph structure without overwhelming complexity.

1) Training & Loss Function: Neural networks begin with nodes that represent the input data (or input features) and process this information through multiple layers, progressively transforming and evaluating it. Eventually, the network produces an output that can be used for tasks such as classification (predicting discrete values), regression (predicting continuous values), or generation (creating new realistic data). A common evaluation metric for classification tasks is accuracy, which simply measures the proportion of correctly classified nodes out of all predictions.

For the model to perform well, training is required by adjusting the weights in the layers. The weights do not need to have perfect values, but the better values they have, the better the model will perform. Mathematically, the loss function is defined to quantify how many mistakes the model makes when trying to classify nodes, and then this function is optimized to find better weights. The core of optimization in neural networks (and optimization in general) is Gradient Descent [5]. Here the gradient is calculated using all of the data (and points toward or away from the direction in the parameter space where better values for the weights exist). Finding and using this gradient can be very computationally expensive for larger neural networks. To address this limitation, Stochastic Gradient Descent (SGD) can be used instead [6], which only calculates the gradient based on one randomly selected data point. This is then repeated for other randomly picked data points until the entire dataset has been used. When the entire dataset has been used for training, an epoch is said to have passed. Mini-batch SGD is a middle ground between these two approaches, taking a number of data points (a mini-batch) and calculating the gradient based on these data points. Again, this is repeated until all data points have been used and the process starts over. The goal in training is to minimize the mistakes the model makes on the training dataset. The overall objective is to minimize the mistakes the model makes on the test dataset.

The simplest loss function for classification is the 0-1 loss function which simply adds 1 if the prediction is wrong and adds 0 if it is correct. This function is not easy to optimize since it lacks a gradient and does not indicate how confident the model is in its predictions. A more commonly used loss function is the cross-entropy loss, which provides probabilities that the data should be classified as different labels, after which the label with the highest probability is chosen [7]. This way, the model is rewarded if it selects the correct label with a high probability, slightly penalized if it picks the correct label but was not fully certain in its prediction, and punished harshly if it assigned the correct label a low probability of being correct. To compute cross-entropy loss, a softmax function is usually first applied to the model's raw outputs. The softmax transforms the raw outputs (often called logits) into probabilities that sum to one across all classes. In practice, it is often more stable to apply the log of the softmax (called log-softmax) and then use a loss function designed for log-probabilities. Mathematically, applying log-softmax and then using a negative log-likelihood loss is exactly the same as applying cross-entropy loss directly. This method improves numerical stability, especially when dealing with very large or very small numbers during training, and is commonly used in modern implementations.

2) Over & Underfitting: Even if perfect performance is achieved in training with 100% accuracy there, the accuracy on the test dataset can still be quite poor. Consider data from the function $f(x) = 3x^2$ with some noise, where the goal is to find the underlying function from these data points. Because of this noise, it cannot be immediately determined that it is from $f(x) = 3x^2$. If 50 data points are available in the training dataset, then the function that would give

zero loss would be a function of the form $g(x) = cx^{49} + b$ which would perfectly interpolate all data points. However, when this function is applied to the test dataset with new points, the function would yield poor accuracy results. This is a phenomenon known as overfitting which occurs when the model is allowed to be too flexible and training continues for too long. The opposite would be seeking a function of the form $g(x) = cx + b$ which is not expressive enough to represent the original function, regardless of training duration. This is known as underfitting. It is common to allow the model to be more flexible but then penalize it the more expressive it becomes. Methods like this, that aim to reduce overfitting (and thus improve the accuracy on the test dataset) are known as regularization [8]. This can be implemented by adding a term in the loss function that grows larger the more complex the model becomes. Another widely used method of regularization is called dropout [9]. A parameter between zero and one is set, for example $p = 0.4$. Then during training, every node has a 40% chance to be removed from the neural network. One way to understand why this would work is that certain nodes may represent certain properties in the data. If those features are overrepresented in the training dataset then the network may start relying too heavily on those features. By dropping those nodes, the model is forced to utilize other properties. Another more general example is to reduce co-adaptation. If one node makes a mistake, but then the next node tries to correct this mistake, it would be preferable for the first node to be correct immediately. By dropping random nodes, co-adaptation is disrupted and simpler relations are enforced.

3) Optimizer: With many optimizers, including basic Stochastic Gradient Descent, the same learning rate is applied to all parameters and stays constant unless manually adjusted. However, different parameters can behave very differently during training. Some might require large updates, while others should move carefully. Without delving into extensive mathematical details, the optimizers used in this project are discussed in this section.

RMSprop (short for Root Mean Square propagation) is an optimizer designed to adapt the learning rate individually for each parameter based on how the gradients behave [10]. It keeps a running average of the squared gradients for each weight, but it does not accumulate them forever. Instead, it uses an exponentially decaying average, meaning that more recent gradient information matters more than very old gradients. This way, if a parameter consistently has large gradients, RMSprop will lower its learning rate to avoid instability, and if the gradients are small, it keeps the learning rate larger to allow faster learning.

Building on this idea, Adam (short for Adaptive Moment Estimation) improves optimization further [11]. Like RMSprop, Adam keeps a running average of the squared gradients to adapt learning rates, but it also keeps a separate running average of the gradients themselves (and not just their squares) to calculate the momentum. If the parameter set is constantly being pushed in a certain direction, momentum will increase the learning in this direction. Adam combines ideas from both momentum (smoother updates based on past gradients) and adaptive learning rates (adjusted based on the variability of the gradients).

However, the regularization technique known as weight decay (which shrinks the weights slightly after each training step) is not fully compatible with Adam. If weight decay is applied by simply adding it into the loss function, the optimizer's internal adaptation mechanisms also modify how weight decay behaves, leading to unpredictable effects. The optimizer AdamW separates weight decay completely from the optimization step [12]. This should lead to better generalization and more predictable training behavior.

B. Graph Neural Networks

In this subsection, the foundations of Graph Neural Networks are introduced, including how graphs are used in machine learning, the principles of message passing, and finally the architecture of Graph Convolutional Networks (GCNs) which are used in this project.

To represent graphs in machine learning, an adjacency matrix A is commonly used. If node u has an edge toward node v, then $A[u, v] = 1$; otherwise, $A[u, v] = 0$ [13]. In some cases, edges can also carry weights, meaning $A[u, v]$ could take a value other than 0 or 1, depending on the strength of the connection. For large graphs, the adjacency matrix is typically very sparse, with most entries being zero. In addition to the structure captured by the adjacency matrix, nodes often have associated feature vectors that describe their attributes. These features are collected into a feature matrix X, where each row corresponds to a node's features. When applying machine learning to graphs, it is crucial to combine information from the graph structure A and the node features X in a meaningful way. Graph Neural Networks (GNNs) have become the dominant modern approach for this, enabling models to learn directly from both structure and features to perform tasks such as node classification, link prediction, and graph classification [14]. In practice, it is also common to normalize the adjacency matrix when performing computations on graphs. This helps avoid issues where nodes with many neighbors dominate the aggregation process.

1) Message Passing: Graph Neural Networks (GNNs) share much of their theory with regular neural networks, but with one very important addition of message passing. Message passing can be simply understood as nodes exchanging information if they have an edge connecting them. In every layer of a GNN, there is first a round of message passing where each node aggregates information from itself and its neighboring nodes to create a new representation (or "message") which encodes information about the node's local neighborhood [2]. Each node then sends this message to its neighbors and simultaneously receives messages from them. After aggregation, each node updates its internal state using the received information. This sequence of message passing and updating is repeated in every layer of the network.

Mathematically, message passing at a single layer can be broken down into an aggregation step, where each node collects and summarizes information from its neighbors (and often also from itself), and an update step, where the aggregated information is transformed into a new node feature,

typically by applying a linear transformation followed by a nonlinearity.

A simple and widely used example of message passing is the Graph Convolutional Network (GCN), which uses the concept of a discrete convolution to aggregate the information in message passing. In a GCN, the layer-wise update rule for all nodes can be expressed compactly in matrix form as:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right)$$

Here, $H^{(l)}$ denotes the matrix of node features at layer l (with $H^{(0)} = X$ being the input features), and $W^{(l)}$ is the trainable weight matrix at layer l. The matrix $\tilde{A} = A + I$ is the adjacency matrix of the graph with added self-loops, which ensures that each node can pass information to itself. The matrix $\tilde{D}$ is the diagonal degree matrix of $\tilde{A}$, with each diagonal element $\tilde{D}[i,i]$ equal to the sum of the i-th row of $\tilde{A}$. The normalization with $\tilde{D}^{-1/2}$ on both sides symmetrically adjusts the scale of the aggregated features, which helps prevent numerical instability during training. Finally, σ is a nonlinear activation function, typically chosen to be the ReLU function. [15]

To intuitively understand this formula, one can recognize that $\sigma\left(H^{(l)}W^{(l)}\right)$ is basically analogous to the normal neural network update equation in section 2.1. The main difference is the multiplication by $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ before activation, where $\tilde{A}$ indicates how the nodes are connected (and how the information $H^{(l)}W^{(l)}$ should be shared). The $\tilde{D}^{-1/2}$ terms scale that matrix so nodes with many neighbors do not obtain high feature values (and therefore certain representations) simply because they have many neighbors.

2) Other Types of GNNs: There are other versions of GNNs that are not used in this project but are worth mentioning. While GCNs treat all neighboring nodes equally during aggregation, other GNN variants modify this behavior. For example, in Graph Attention Networks (GATs), the model learns different weights for different neighbors [16], allowing it to pay more attention to certain nodes depending on their importance. This can be beneficial when not all edges are equally informative. Another variant is GraphSAGE (Sample and Aggregate), where instead of aggregating over all neighbors, a random subset of neighbors is sampled at each layer [17]. This greatly reduces computational costs and makes GraphSAGE particularly suitable for very large graphs where full neighbor aggregation would be impractical.

C. Dataset and Tools

This subsection outlines the tools and dataset used in the experiments, specifically focusing on the Cora citation dataset and the PyTorch Geometric library.

One of the most widely used datasets for Graph Neural Networks (and the one employed in this study) is the Cora dataset, which consists of roughly 2700 scientific papers with roughly 5400 total citations between the papers [3]. Each paper is represented as a node, and if there is a citation between two papers, a corresponding edge is added between those nodes. Every paper is classified into one of seven categories. Each paper (node) also has a vector with roughly 1400 elements, each one relating to a word. If the word exists in the paper, the element in the vector takes the value one, and zero otherwise.

PyTorch Geometric (PyG) is a library built on top of PyTorch, designed specifically for working with graph-structured data. It provides efficient implementations of various GNN layers like GCNConv and GATConv, along with tools for data loading, batching, and graph-specific operations like neighborhood sampling [18]. PyG handles the underlying message-passing operations and sparse matrix computations efficiently, making it well-suited for larger graphs. It is also worth noting that PyG is very user-friendly with most advanced procedures built in and is designed to be easy to use. However, this naturally comes with a trade-off for flexibility where there is less total control of what the model can do. Therefore, it is not possible, for example, to use regular Gradient Descent as an optimizer in a simple way, as the library designers prefer SGD over basic gradient descent for performance reasons. This is often beneficial but since performance comparison is desired, this limitation restricts the available options.

III. Method

In this section, the implementation and evaluation approach for Graph Neural Network (GNN) models is outlined. The objective was to investigate how model-architectural and training-related choices affect performance on a node classification task using the Cora dataset and compare the results with the performance of other implementations in the literature.

A. Dataset and Preprocessing

The Cora dataset was utilized, consisting of a citation network containing 2,708 nodes (papers) and 5,429 edges (citations). Each node is associated with a binary feature vector of length 1,433 indicating the presence of specific words in the paper. Each node also has a label corresponding to one of seven paper topics.

Standard practice was followed by using the Planetoid data split, where the dataset is divided into predefined training, validation, and test sets. The dataset was preprocessed using PyTorch Geometric's built-in loader, which ensures the feature matrix, labels, and data masks are loaded consistently with the established split.

B. Base Model Architecture and Implementation

A standard Graph Convolutional Network (GCN) was employed as the base model, implemented in PyTorch Geometric. The base model consists of two GCN layers with ReLU activation and an output layer with log-softmax for classification. Each GCN layer performs message passing, aggregating node feature information from neighbors and updating node representations. Cross-entropy loss was used for training and performance was evaluated using classification accuracy. The optimizer for the base model is SGD with a weight decay of 0.001. To find an appropriate learning rate for each model configuration, the learning rate was selected from the options $0.01, 0.05, 0.1, 0.2$ based on validation set performance after 200 epochs. Subsequently, the model was retrained for 200

epochs with the best learning rate while tracking the training loss as well as the performance on the training, validation, and test datasets.

C. Experimental Design

Three sets of experiments were conducted to investigate the effect of different hyperparameters. For each experiment, the base model was used and only one specific hyperparameter was varied:

- **Number of Layers:** Models were trained with 2, 3, 4, and 5 GCN layers to observe how deeper networks affect performance.
- **Dropout Rate:** Different levels of dropout were applied between layers to test its effect as a regularization technique. Models were tested with dropout probabilities $0, 0.2, 0.4, 0.6,$ and 0.8.
- **Optimizer Choice:** The performance of four different optimizers was compared: SGD, Adam, AdamW, and RMSProp.

For each configuration, an appropriate learning rate was first determined as previously described, and then the model was trained for 200 epochs while recording accuracy on the training, validation, and test sets every 20 epochs.

D. Evaluation Metrics

The main evaluation metric was test set accuracy (and final test set accuracy). Training and validation accuracy were also tracked to monitor convergence behavior and identify signs of underfitting or overfitting.

IV. Results

The experimental results are presented through a series of plots examining different aspects of model behavior and performance. Initially, the frequency distribution of features is illustrated in Fig. 1, followed by the dataset split distribution in Fig. 2. For the base model analysis, the validation accuracy as a function of learning rate is shown in Fig. 3. Subsequently, the training loss progression over epochs is presented in Fig. 4, and the accuracy evolution over time for the training, validation, and test datasets is displayed in Fig. 5.

Regarding the layer variation experiments, the final accuracy for each number of layers is first presented in Fig. 6, followed by the test accuracy evolution over time for different layer counts in Fig. 7. For the dropout rate investigation, the final accuracy for each dropout rate is shown in Fig. 8, and the corresponding test accuracy progression over time is illustrated in Fig. 9. For the optimizer comparison, the final accuracy for each optimizer is presented in Fig. 10, with the test accuracy evolution over time for different optimizers shown in Fig. 11.

The analysis concludes with a comprehensive summary of the three experiments, focusing exclusively on the final test dataset accuracy. The layer experiment summary is presented in Fig. 12, followed by the dropout rate comparison in Fig. 13, and finally the optimizer choice comparison in Fig. 14.

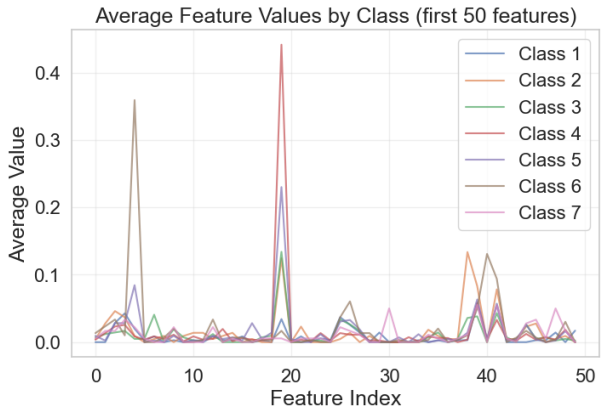


Fig. 1. Plot showing how often the first 50 features (words) appear in a node (paper) on average

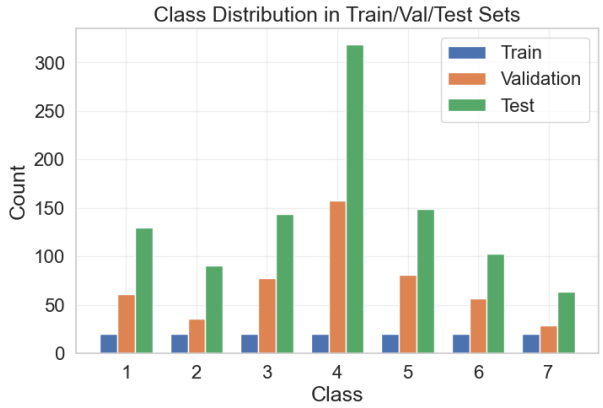


Fig. 2. Plot showing the class- and data-split distribution for the split of the Cora dataset

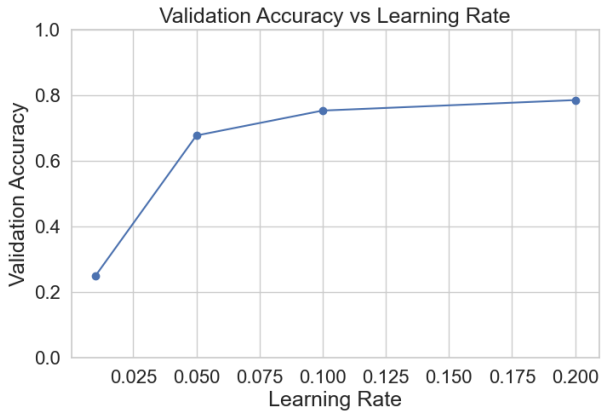


Fig. 3. Plot for the base model showing the final validation accuracies for each learning rate after the full 200 epochs

V. Discussion

In this section, the experimental results are analyzed and discussed systematically. The analysis begins with the performance of the base model, followed by examination of the three separate experiments, and concludes with overall observations as well as discussion of limitations and future work.



Fig. 4. Plot for the base model showing the training loss for the model over time

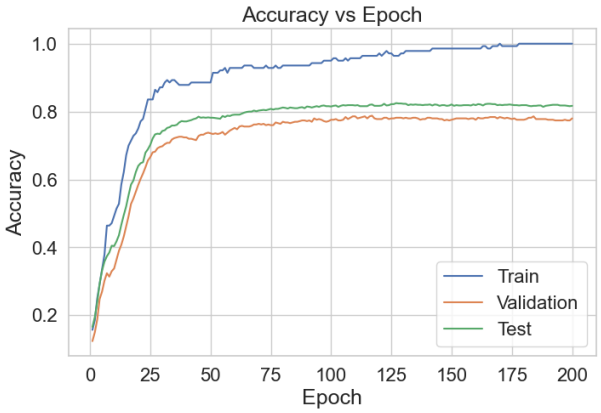


Fig. 5. Plot for the base model showing the training-, validation- and test accuracy over time

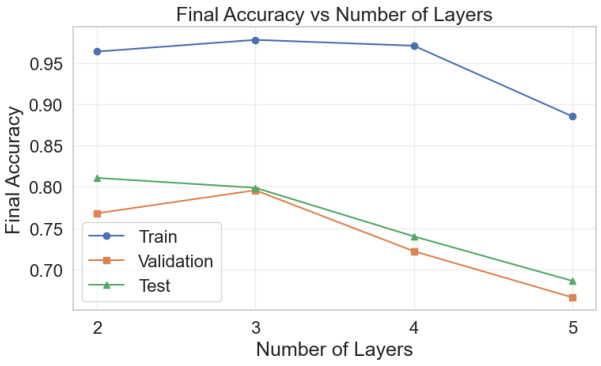


Fig. 6. Plot for the layer experiment showing the final training-, validation- and test accuracy for each model with different layer count



Fig. 7. Plot for the layer experiment showing the test accuracy for each model with different layer count over time (recorded once every 20th epoch)

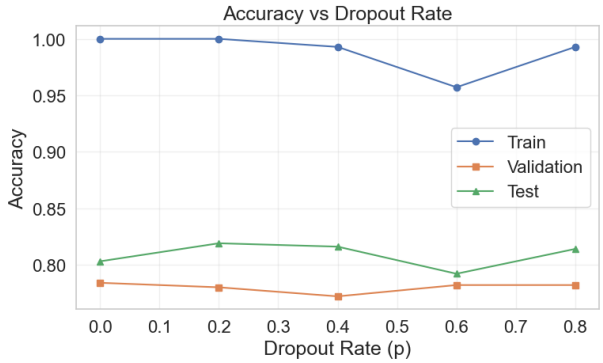


Fig. 8. Plot for the dropout experiment showing the final training-, validation- and test accuracy for each model with different dropout rate

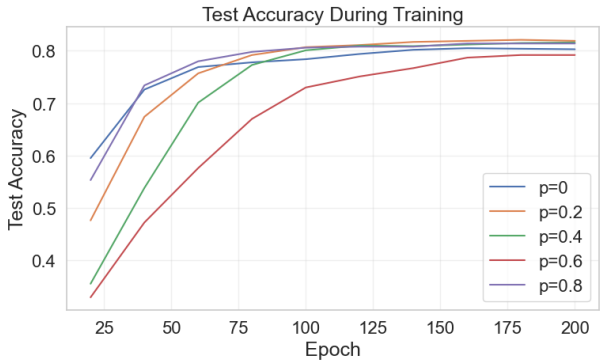


Fig. 9. Plot for the dropout experiment showing the test accuracy for each model with different dropout rate over time (recorded once every 20th epoch)

A. *Base model performance*

Analysis of the base model reveals that a learning rate grid search demonstrated that higher learning rates generally led to better validation accuracy, with optimal performance achieved at a learning rate of 0.2 in Fig. 3, yielding a validation accuracy of 78.7%. Training the model with this learning rate resulted in a final test accuracy of 81.2% after 200 epochs. The training loss in Fig. 4 appears reasonable at first glance, but considering the number of epochs, training progresses slowly, suggesting that the optimizer or learning rate selection may be suboptimal. In Fig. 5, it is observed that the training accuracy became near perfect towards the end, while the test and validation accuracy stagnated, indicating overfitting. It is also noted that the validation accuracy is consistently lower than the test accuracy, which deviates from expectations.

B. *Effect of varying number of layers*

In Fig. 6, test accuracy decreases steadily as the number of layers increases, with the lowest performance observed at five layers. This is likely due to oversmoothing, where increased

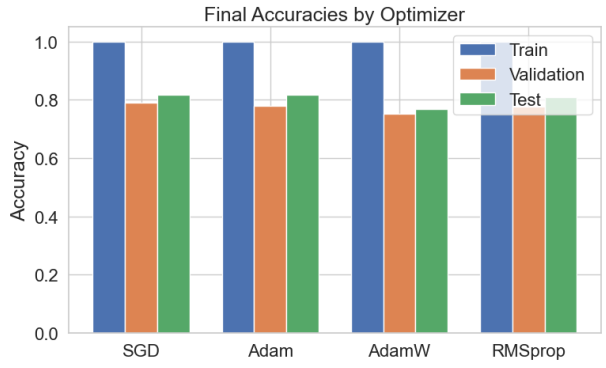


Fig. 10. Plot for the optimizer experiment showing the final training-, validation- and test accuracy for each model with different optimizer



Fig. 11. Plot for the optimizer experiment showing the test accuracy for each model with different optimizer over time (recorded once every 20th epoch)

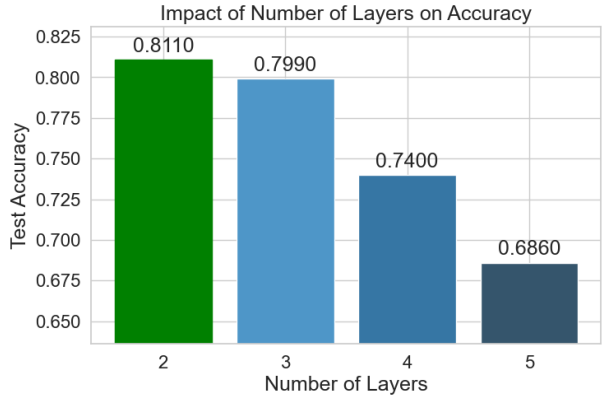


Fig. 12. Summarizing plot for the layer experiment comparing final test accuracy for each model

message passing leads to over-homogenization of node features, reducing discriminative power. The further the message has to travel through citations, the weaker the relevance to the target node becomes. Interestingly, for three layers, the test accuracy does not deteriorate substantially, suggesting potential for effective models at that depth. Again, validation accuracy is lower than test accuracy. Fig. 7 shows that five layers require over 100 epochs to reach even 50% accuracy. While three layers appear to outperform two layers during most of training, repeated simulations show that this does not

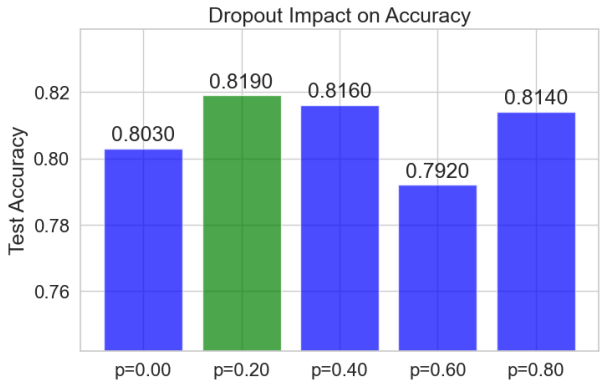


Fig. 13. Summarizing plot for the dropout experiment comparing final test accuracy for each model

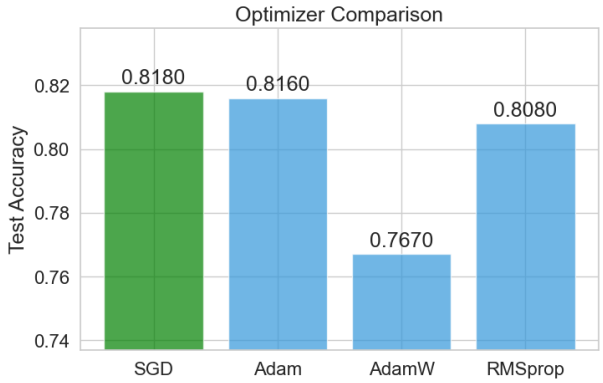


Fig. 14. Summarizing plot for the optimizer experiment comparing final test accuracy for each model

occur consistently, instead likely reflecting the randomness of SGD affecting the results.

C. Effect of dropout

In Fig. 8, the final accuracy remains largely unaffected by the dropout rate. Even when dropping 80% of nodes during training, performance remains similar to training without dropout. The highest test accuracy of 81.9% was observed at $p = 0.2$, but all values were similar. This indicates that for the base model and dataset employed, model complexity and regularization are already well balanced, and varying dropout had limited impact. Fig. 9 provides limited insight as attempts to establish a correlation between dropout rate and test accuracy from the results fail, since the best-performing dropout rates at the end of training were $p = 0.2$, $p = 0.4$ and $p = 0.8$, while $p = 0$ and $p = 0.6$ lagged behind. There is no logical explanation for why these particular dropout rates would be different, therefore again suggesting randomness in the experiment, likely stemming from the use of SGD.

D. Effect of different optimizers

In Fig. 10, final accuracy remains mostly unchanged across different optimizers, with all models achieving high training accuracy and comparable test performance. Interestingly, SGD ultimately performed best. Fig. 11 provides critical

context: SGD required approximately 100 epochs to converge, compared to much faster convergence by other optimizers. Importantly, SGD used the highest learning rate (0.2), while the other optimizers used lower learning rates (0.01 and 0.05), highlighting an inconsistency in learning rate ranges. This suggests that the comparison between optimizers may not be fully fair as it is possible that optimal performance is achieved with higher learning rates for SGD and lower learning rates for the others. Moreover, the fixed weight decay of 0.001 may not be optimal across optimizers and should be fine-tuned. Notably, AdamW consistently performed worse than RMSProp and Adam, when it theoretically should be superior, again suggesting that the weight-decay hyperparameter also needs to be carefully chosen.

E. Overall observations

Across all experiments, the base model demonstrated strong and stable performance, with test accuracy varying only marginally despite significant changes in architecture and training strategy. This robustness suggests that the model performs well on the Cora dataset, and that hyperparameter adjustments yield incremental improvements. However, it is crucial to acknowledge several experimental limitations that reduce the reliability of certain conclusions. These include a fixed and potentially suboptimal weight decay, a narrow learning rate range, and the use of SGD with 200 training epochs which represents an unrealistic setup in many practical contexts. While the goal was to explore the effects of design choices rather than to optimize performance, using SGD in the base model may still have been a suboptimal choice. Its slow and unreliable convergence limits the ability to draw conclusions from training curves, as they may reflect stochastic variance rather than genuine performance differences.

Despite these issues, some consistent patterns emerge. Overfitting is persistent across settings, with training accuracy nearing perfection while validation and test accuracy stagnate. The limited impact of dropout, even at high rates, suggests that regularization is either already sufficient or ineffective given the base model or the dataset's characteristics. The frequent observation that test accuracy exceeds validation accuracy could point to a biased or unbalanced data split.

This data split warrants closer attention. Fig. 1 shows a sensible word distribution, with general and domain-specific terms as expected. However, Fig. 2 reveals a surprising imbalance with only 20 training nodes per class, a small and uneven sample relative to the much larger validation and test sets. This unexpected distribution represents the Planetoid split which multiple other papers use for benchmarking. This split naturally creates challenges compared to using an organic dataset split. Despite these challenges, achieving up to 81.9% accuracy in a seven-class classification task is impressive. The Planetoid split remains popular for benchmarking, allowing comparisons to prior work such as the original GCN paper [15], which achieved 81.5%. Hence, the best-performing model at 81.9% is competitive despite minimal hyperparameter tuning. It should be noted that selecting the best performing configuration retrospectively can introduce bias, so this performance should not be overstated. Nonetheless, state-of-the-art models on Cora only reach about 90% accuracy [19], suggesting that significant improvement is possible with proper tuning and innovative design choices.

F. Limitations and future work

While this study explored a range of architectural and hyperparameter settings, several improvements remain necessary. Most notably, expanding the search space for learning rates and other hyperparameters such as weight decay or hidden dimension size would provide more robust conclusions. Training for 200 epochs was excessive, especially since models often converged much earlier. Future studies should use shorter training schedules, as prolonged training added limited value apart from revealing the slow convergence of SGD. Even if SGD suffered from a suboptimal learning rate, the results suggest that modern optimizers are preferable, offering faster convergence and better performance in early stages. It is believed that further performance gains are more likely to come from improved design decisions than extended training. The experiments demonstrated that many optimizers perform strongly well before 200 epochs, underscoring the unnecessary nature of prolonged training.

VI. Conclusion

Through this project, the foundational concepts behind Graph Neural Networks (GNNs) and their implementation have been explored. The investigation encompassed their connection to standard neural networks and how they incorporate graph structure through message passing. Key training techniques, optimization strategies, and regularization methods that influence model performance were also reviewed.

Multiple models with varying design choices were implemented and evaluated. The results demonstrated that increasing the number of layers negatively impacted performance, while choices such as optimizer selection and dropout utilization did not exhibit large effects on classification accuracy. However, numerous reasons for why these experiments were likely flawed have been discussed in the report. This highlights the importance of mindful decision making when training GNNs and the many pitfalls that exist in this domain. Nevertheless, the study demonstrates that simple hyperparameter tuning can achieve very good performance by utilizing built-in tools in PyTorch Geometric combined with a basic understanding of GNN concepts.

Ultimately, this project underscores both the promise and the challenges of working with GNNs. As graph-based data becomes more prevalent across domains, understanding how to build and refine GNN models will become an increasingly valuable skill.

Key findings:

- Increasing the number of layers in a GCN negatively affected performance. When selecting an optimizer, advanced optimizers should be preferred over SGD due to training efficiency considerations. Results regarding the correlation between dropout rate and accuracy were inconclusive.

- Even simple GCN models, when paired with effective tools like PyTorch Geometric, can achieve competitive performance with minimal tuning.
- Experimental outcomes are highly sensitive to architectural design and training setup, underscoring the need for careful design considerations when developing GNNs.

VII. Acknowledgment

The authors would like to thank their supervisors, Zeyang Li and Carlo Fischione, for their feedback and help throughout the project.

Appendix A
Jupyter Notebook

Jupyter Notebook containing all code used to generate results and plots.

References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Massachusetts Institute of Technology, Cambridge, Massachusetts: MIT Press, 2016, http://www.deeplearningbook.org.

[2] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, vol. 70. Sydney, Australia: PMLR, 2017.

[3] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, and T. Eliassi-Rad, "Collective classification in network data," *AI Magazine*, vol. 29, no. 3, p. 101, 2008.

[4] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 315–323.

[5] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[6] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proceedings of the 19th International Conference on Computational Statistics (COMPSTAT)*. Paris, France: Springer, 2010.

[7] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. Massachusetts Institute of Technology, Cambridge, Massachusetts: MIT Press, 2012.

[8] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY: Springer, 2006.

[9] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[10] T. Tieleman and G. Hinton, "Lecture 6.5 - rmsprop: Divide the gradient by a running average of its recent magnitude," Coursera: Neural Networks for Machine Learning (Lecture 6.5), 2012.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[12] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, New Orleans, Louisiana, United States, 2019.

[13] W. L. Hamilton, *Graph Representation Learning*. McGill University, Montreal, Canada: Morgan & Claypool, 2020.

[14] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, 2020.

[15] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2017.

[16] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, Vancouver, Canada, 2018.

[17] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[18] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, New Orleans, Louisiana, United States, 2019.

[19] (2025, Apr.) Node classification on cora. [Online]. Available: https://paperswithcode.com/sota/node-classification-on-cora